COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	10/13/2026	**DATE SUBMITTED:**	10/13/2026 (Procedure) 10/20/2026 (Supplementary)
NAME:	Cabug, Jeffordniel H. Salcedo, Lance Cristopher S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Quick Sort	Although Quick Sort is a divide-and-conquer algorithm, it's considered one since it picks a pivot first, then it divides itself, since it also needs a pivot to work, it recurses itself until it's sorted.
Brute Force	Bubble Sort	It is a brute force algorithm since it repeatedly compares adjacent elements and swaps them if they are not in the right order, it continuously does this until the array is sorted.
Backtracking	Depth-First Search	Because it relies heavily on LIFO (Last in, First Out or stack). When a DFS explores, it goes into a path until its a dead end, it will pop itself and make a new direction exploring new branches.
Greedy	Selection Sort	It is a greedy algorithm since it chooses the smallest element from the unsorted part of the array and it also makes the best immediate choice at each step to make it sorted.
Divide-and-Conquer	Merge Sort	It uses a divide-and-conquer algorithm since it divides into two equal halves, then becomes smaller through conquering it and combining it once more making it sorted.

Table 12-1. Algorithmic Strategies and Examples

Screenshot	

main.cpp > getMinSteps(int)

```cpp
#include <iostream>
#include <algorithm>
#include <iterator>

int memo[1001];

int getMinSteps(int n)
{
    // Base case
    if (n == 1) return 0;

    // Return the saved result if already solved
    if (memo[n] != -1) return memo[n];

    // Subtract 1
    int r = 1 + getMinSteps(n - 1);

    // Divide by 2 if divisible
    if (n % 2 == 0)
        r = std::min(r, 1 + getMinSteps(n / 2));

    // Divide by 3 if divisible
    if (n % 3 == 0)
        r = std::min(r, 1 + getMinSteps(n / 3));

    // Save the result
    memo[n] = r;

    return r;
}

int main()
{
    int n;

    std::cout << "Enter a positive Integer: ";
    std::cin >> n;

    if (n < 1 || n > 1000)
    {
        std::cout << "Please enter a number from 1 to 1000.";
        return 0;
    }

    std::fill(memo, memo + 1001, -1);

    std::cout << "Minimum steps: " << getMinSteps(n) << std::endl;

    return 0;
}
```

OUTPUT:
// Given n is 10
Outcome is 3

```
Enter a positive Integer: 10
Minimum steps: 3
```

Analysis	We first used the header files <algorithm> and <iterator>. <algorithm> is used to provide the min() function, in the int memo[1001] it was used to store array elements from 0 to 1000. Now in the getMinFunction(), it first uses a recursion algorithm, if (n == 1) it stops, if (memo[n] != -1) checks if it was solved, it continues if it doesn't. The following line subtracts 1, divides 2 or 3 if it was divisible, then saves the result. In the main function, it uses int n, then a display message said to enter a positive integer, it uses std::cin to interact. The for loop will not work to check if the number entered is within the range of 1 to 1000 then it returns. It will now initialize the memo, then it calculates how many minimum steps were needed.

Table 12-2. Memoization Implementation

Screenshot

mainBottomUp.cpp > getMinSteps(int)

```cpp

#include <iostream>
#include <algorithm>

int memo[1001];

int getMinSteps(int n)
{
    // Base case
    if (n == 1)
        return 0;

    // Return the saved result
    if (memo[n] != -1)
        return memo[n];

    // Subtract 1
    int r = 1 + getMinSteps(n - 1);

    // Divide by 2 if divisible
    if (n % 2 == 0)
        r = std::min(r, 1 + getMinSteps(n / 2));

    // Divide by 3 if divisible
    if (n % 3 == 0)
        r = std::min(r, 1 + getMinSteps(n / 3));

    // Save the result
    memo[n] = r;

    return r;
}

int main()
{
    int n;

    std::cout << "Enter a positive integer: ";
    std::cin >> n;

    if (n < 1 || n > 1000)
    {
        std::cout << "Please enter a number from 1 to 1000.\n";
        return 0;
    }

    std::fill(memo, memo + 1001, -1);

    std::cout << "Minimum steps: "
              << getMinSteps(n) << '\n';

    return 0;
}
```

OUTPUT:
// Given n is 10
Outcome is 3

```
Enter a positive integer: 10
Minimum steps: 3
```

Analysis

In this Bottom-up Dynamic, it uses an integer array dp[1001] to store the minimum number of steps needed to reduce each number to 1. It starts with the base case dp[1] = 0 because no steps are needed when the number is already 1. The for loop processes each number from 2 up to the input number n. For every number, the program first checks the steps needed when subtracting 1. If the number is divisible by 2 or 3, it also considers dividing by those numbers and uses std::min() to select the option with the fewest steps. After calculating the results, the program returns dp[n], which represents the minimum number of steps for the given input. When the input is n = 10, the output is Minimum steps: 3 because the shortest sequence is 10⟶9⟶3⟶1. The code has a time complexity of O(n) because it processes each number once, and a space complexity of O(n) because it uses an array to store the calculated results.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE:

```
#include <iostream>
#include <vector>

int M, N, target;
    std::vector<std::vector<int>> mat;
    std::vector<std::vector<std::vector<int>>> memo;

int dp(int i, int j, int cost){
    if (cost < 0) return 0;
    if (i == 0 && j == 0)
        return (cost == mat[0][0]) ? 1 : 0;

    if (memo[i][j][cost] != -1)
        return memo[i][j][cost];

    int res = 0;
    if (i > 0) res += dp(i-1, j, cost - mat[i][j]);
    if (j > 0) res += dp(i, j-1, cost - mat[i][j]);

    return memo[i][j][cost] = res;
}
```

```
int main(){
    mat = {
        {4,7,1,6},
        {5,7,3,9},
        {3,2,1,2},
        {7,1,6,3}
    };
    target = 25;
    M = mat.size();
    N = mat[0].size();

    memo.assign(M, std::vector<std::vector<int>>(N, std::vector<int>(target+1, -1)));

    std::cout << "Result: " <<dp(M-1, N-1, target) << std::endl;
    return 0;
}
```

OUTPUT:

```
Result: 2
```

ANALYSIS

SALCEDO - The code we used is a top-down approach with memoization. It calls itself (or recursion) to reach both cells i and j with exactly cost remaining. Each cell subtracts with the cell's value from cost and recurses it (both above and left) and it shows the results. Memoization is used here to solve many subproblems many times and can reach multiple different paths. The result is 2 because there are only 2 paths that can be solved that are cheap.

CABUG - The code uses recursion to find the possible paths in the grid based on the given cost. It checks the cells above and to the left while subtracting their values from the remaining cost. It also uses memoization to save the results of previous calculations so that the program does not have to repeat the same process. After checking the possible paths, the program counts the paths that meet the required cost. The output is 2, which means there are two possible paths that satisfy the given condition. Through this program, we can see how recursion and memoization work together to solve a problem more efficiently.

Table 12-4. Supplementary

CONCLUSION:

CABUG:
In this activity we learn the application of different algorithmic strategies, including recursion, brute force, backtracking, greedy algorithms, and divide-and-conquer. Through the implementation of memoization and bottom-up dynamic programming, I was able to apply the concepts of recursion, storing previously calculated

results, and solving problems step by step. Despite the challenges encountered during coding, the activity allowed me to apply these concepts in solving programming problems and determining efficient solutions.

SALCEDO:
We were able to finish the hands-on activity. I also learned on what type each sorting algorithm is like Bubble Sort being a Brute Force Strategy since it repeatedly compares itself. The procedure part was understandable but I was having a hard time on the codes on both procedure and supplementary. Overall, the activity served us as a challenge as a pair on what to do on this hand-on activity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Google Gemini, Brave Leo**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code analysis, idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used these AI tools to know what to do in the supplementary as well as to answer the given questions in the procedure and all of our answers are ours only.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.